GenStream: **Toward Applying Generative Codecs to Video Streaming**

Paper #648

Abstract

In this paper, we present GenStream, the first video streaming system that leverages generative codecs in the resource-constrained transmission environments with limited bandwidth and lossy channels. Traditional video codecs with forward-error correction (FEC) are limited by their low compression efficiency and loss resilience.

GenStream utilizes the recent advances of generative compression to encode the video in a sequence of tokens. This new representation of video offers is more bandwidth-efficient and loss-resilient. To handle the network dynamics, GenStream proposes a practical token adaptation approach that leverages the cross-layer information. Experiments on real-world datasets show that GenStream largely cuts transmitted data volume while remaining robust to packet loss compared to state-of-the-art baselines. These results demonstrate the promise and effectiveness of adopting the generative techniques in a video streaming system, unlocking a new operating point in the efficiency–robustness space for video delivery under extreme network constraints.

1 Introduction

Video streaming accounts for 82% of Internet traffic and has been surging in recent years [5]. With the advanced content delivery network (CDN) deployment and high-speed wireless networks, consumers can enjoy a high-quality video experience nowadays. According to [31], only 4% of video streams experience stall events. Nevertheless, video streaming is challenging yet important in recent emerging applications. For example, satellite communication usually has to capture and send high-resolution video frames of the Earth to the ground for downstream tasks such as forest fire prediction [24, 27]. Underwater robots have to consistently send the captured environment back to the surface for autonomous manipulation [14]. These application scenarios feature a common *extreme transmission environment*: 1) limited bandwidth relative to transmission content; 2) lossy channels that may degrade the video streaming quality.

To address these challenges, the generic solution is to adopt the video compression techniques with a forward error correction (FEC) approach [21]. However, as we will show in §2.2 shortly, either the traditional video codecs such as h.264/265 or recently proposed neural codecs [4, 12] appear to have low compression efficiency. While several techniques, such as frame filtering [13], object detection and segmentation, are also helpful in reducing the bandwidth consumption. They are usually tailored for a specific application, such as traffic analysis, which lacks generalizability. On the other hand, while there have been lots of advancements in FEC [21], the dilemma lies in determining the appropriate amount of redundancy to strike a balance between the efficiency and robustness of FEC. More importantly, FEC is only effective when the packet loss rate remains below its correction threshold.

In this paper, we propose GenStream, a novel video streaming solution that achieves both bandwidth-efficiency and loss-resilience in constrained environments. Inspired by the recent advancements in generative models [2], GenStream adopts the generative codecs to compress a block of video frames into a compact latent representation, referred to as *token*. The codecs are also known as *video tokenizers*. Those tokens are then transmitted over the lossy network. Even without FEC codes and in the presence of the packet loss, the receiver can still effectively recover the video gracefully by leveraging the "generative" capability of the model.

Directly applying the video tokenizer in the video streaming system is non-trivial, as network dynamics such as fluctuating bandwidth and varying packet loss patterns should be considered. To address this challenge, GenStream proposes a practical two-step token adaptation approach. First, GenStream decides an initial token number based on the existing well-established rate control algorithms (e.g., GCC [28]). Such algorithms determine the sending rate based on the network statistics like packet delay and packet loss, which in turn instruct the video tokenizer for video compression. However, the initial token number is only determined by the packet-level statistics without considering any context information. Second, we further fine-tune the initial token number based on the frame-level information. Intuitively, given a certain packet loss, videos with more complex and dynamic content require more tokens. To achieve this, we propose a light-weight neural network that integrates the frame content together with packet loss patterns to predict the proper token number.

We evaluate GenStream on an emulated environment with real-world bandwidth data trace [8]. The experimental results show that GenStream improves the video quality, measured by Learned Perceptual Image Patch Similarity (LPIPS), by 38%-59% under similar bitrates, compared to the existing solutions.

In summary, this paper presents GenStream, the first work that leverages the video tokenizer for bandwidth-efficient and loss-resilient streaming. GenStream proposes a practical token adaptation approach that integrates both frame-level and packet-level knowledge to handle the dynamics of network conditions. The effectiveness of GenStream paves the

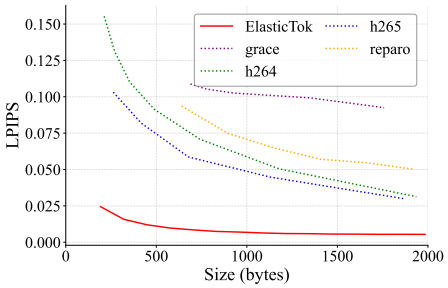


Figure 1: Comparisons of different codecs: video tokenizer has the best encoding efficiency.

way to apply video tokenizer to those resource-constrained transmission environments.

2 Background and Motivations

2.1 Existing Work

Existing video codecs. Traditional video codecs such as h264/265, VP8, and VP9 are important in video streaming applications. These codecs compress video frames using block-based motion prediction, separating them into key frames (I-frames) that are compressed independently and predicted frames (P-/B-frames) that are compressed based on differences between adjacent frames. In recent years, neural network-based codecs [6, 11] have been proposed with the potential to surpass the classic video codecs in coding efficiency, i.e., video quality per data size. In addition, some work like Grace [4] improves the robustness of video codecs [15] by fine-tuning them over different simulated packet losses. However, all of these video codecs compress video at the pixel level, resulting in low compression efficiency for resource-limited environments.

FEC. Forward error correction (FEC) is the primary solution to reduce the impact of packet loss. FEC adds parity packets to the sent packets, which enables the receiver to recover video frames even if some packets are missing. For instance, block codes such as Reed-Solomon code or fountain code have been used in real-world video conference systems. Tambur [21] employs streaming codes to handle burst losses of the packets. However, it is hard for FEC to choose the portion of the parity packet to achieve a good tradeoff between efficiency and robustness. While the packet loss rate exceeds its capacity, the video can be undecodable as illustrated in [4]. (20% parity packet can only handle 20% packet loss on average)

2.2 Video Tokenizer

Overview. Recent advancements in deep generative models [2, 10, 17] offer a promising alternative—*generative compression*. Fig. 2 depicts the workflow of generative compression for videos. The encoder, also referred to as *tokenizer*

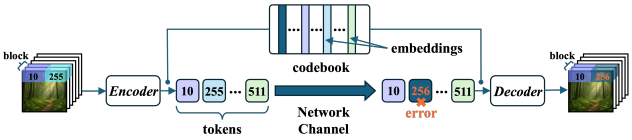


Figure 2: The workflow of generative codecs.

transforms a raw video into a sequence of *tokens*, which represent *indices*[1] of embeddings (*i.e.,* feature vectors) in a pretrained codebook. As illustrated in Fig. 2, each token represents a small patch of *video block* that contains multiple consecutive frames. Unlike previous neural codecs [4] that encode videos as a large set of feature values, this token-based method provides a more bandwidth-efficient representation of semantic information [17, 33]. After the video tokens are transmitted over the network, the decoder, known as *detokenizer*, maps them to their corresponding embeddings and reconstructs the original video.

Generative compression achieves efficient compression with "graceful degradation" [22], where the reconstructed video quality gradually declines as bandwidth decreases or transmission errors occur. Reparo [12] is the first work that applies generative codecs (i.e., VQGAN [7]) to compress the video. However, it simply adopts an image tokenizer and compresses each video frame individually. This leads to a low compression efficiency and frame discontinuity due to the generative nature.

High encoding efficiency. To illustrate the benefits of video tokenizer, we conduct a preliminary experiment and compare the coding efficiency among five different video codecs, including h.264/265 with very-fast preset, traditional codecs, Grace [4], image tokenizer, Reparo [12], and one of the state-of-the-art video tokenizers, Elastictok [32]. We run each codec on a 4090 GPU for two 256x256 test videos from the dataset from [4]. The results are shown in Fig. 1.

The x-axis represents average frame size in bytes, and the y-axis represents LPIPS [34], a learned metric that represents the image similarity. As we can see, Elastictok operates at bandwidth levels beyond the reach of conventional methods. Surprisingly, the coding efficiency of Grace [4] and Reparo [12] is worse than h.264/265. The possible reason for Grace is that it adopts traditional neural codecs which suffer from the out-of-distribution issue when run on new videos. It is explainable for Reparo [12] since it encodes each frame independently.

Optimization opportunity. Most prevailing video/image tokenizers [2, 17] are restricted to encoding videos to a fixed number of tokens irrespective of the visual content of the original video. Over-allocating tokens to the video frames can avoid information loss on complex frames, which wastes

[1]The terms "tokens" and "indices" are commonly used interchangeably.

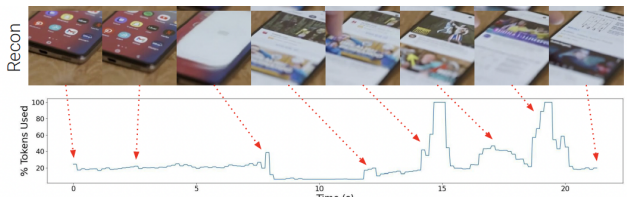


Figure 3: Existing video tokenizer adapts the token numbers based on video contents.

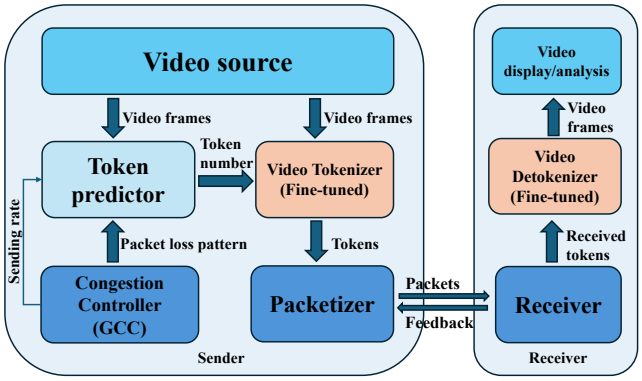


Figure 4: Overview of GenStream.

capacity on simple content. On the contrary, choosing a too small number of tokens yields lossy encodings for the videos with complex visuals. We adopt the state-of-the-art video tokenizer, ElasticTok [32], which dynamically generates the token numbers based on the visual complexity of the video block as shown in Fig. 3. This flexibility offers a new opportunity when we apply it to video streaming: *The token number can be adjusted to compensate for the bandwidth fluctuation and varying packet loss patterns.*

3 GenStream Designs

3.1 System Overview

GenStream leverages and optimizes the existing video tokenizer for video streaming. Fig. 4 shows its framework. At the sender side, the video frames are fed into a *video tokenizer* for compression, which results in a sequence of tokens. These tokens are wrapped into packets, which are subsequently sent over the network at the pace determined by a *rate controller*. At the receiver side, tokens are reconstructed based on the received packets. Some tokens may be missing due to the packet loss. Then, the video detokenizer recovers the video frame with partially lost tokens. Specifically, the video tokenizer and detokenizer are jointly fine-tuned with simulated packet loss to improve their robustness to transmission errors as in [4, 25]. The most important component of GenStream is the *token predictor*, which takes the information of packet-level information and video information as input to adjust the token number.

3.2 Token Adaptation

Deciding the token numbers in a dynamic network environment is challenging. In this paper, we focus on optimizing the video quality during streaming, which would be helpful for downstream tasks like video analytics in the presence of packet loss and limited bandwidth. Intuitively, this can be formulated as a reinforcement learning problem where all environment states can be used to decide the token number to achieve the optimal reward [16, 29]. However, it is hindered by several practical issues that it is hard to collect training data either from real-world or controlled environment which requires interplay of different network layers. Furthermore, reinforcement learning is difficult to converge to an optimal solution and lacks generalizability.

Instead, we propose a more practical approach to determining the token number. At a high level, it contains two steps. First, we determine an initial token number N_{init} based on the sending rate determined by rate control algorithms from the lower layer. For example, in the video conference application, the sending rate is determined by GCC [28] based on the packet delay or packet loss. In a cellular network, the sending rate basically depends on MCS adaptation algorithm and allocated resource blocks [30]. Second, we further adjust the decided token number to capture the interplay between the packet loss pattern and the context information of the video. Specifically, we design a token predictor to predict a corrected token number $N = (1 + \lambda)N_{base}$ on top of N_{base} where $\lambda \in (0, 1)$.

3.3 Token Predictor

Neural Network. We utilize a lightweight neural network as a token predictor, which takes the following metrics as input for compressing k-th video block:

- Sending rate obtained from rate controller r_k.
- Initial token number $N_{init,k}$ decided by sending rate r_k.
- Packet delivery status in the last video block $\mathbf{p}_{k-1} = \{p_{1,k-1}, \ldots, p_{i,k-1}\}$ where $p_{i,k-1}$ is a binary index with 1 meaning the packet i in video block $k-1$ is lost while 0 meaning the packet is delivered.
- Average packet loss $p_{avg,k-1}$ over the last video block.
- Video frames $\mathbf{f}_k = \{f_{1,k}, \ldots, f_{m,k}\}$.

Fig. 5 shows the architecture of the neural network. For the video frames, we use 2D CNN (Convolutional Neural Networks) layers to extract their content features. For each of other scalar and vector values, we add two MLP (Multi-Layer Perceptron) layers. Then, additional MLP layers are used to fuse all outputs from the previous CNN and MLP layers. The probability of all potential token numbers is produced through a softmax layer.

Ground truth and training. The most important thing in training the neural network is to obtain the ground-truth

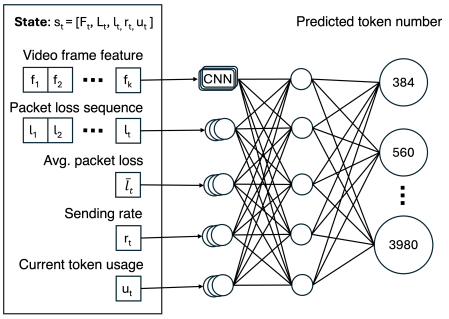


Figure 5: Token predictor.

token number. Our idea is to use the token number which is slightly larger than the initial token number and has the highest *utility*, defined as the improved perceived quality per token, as the ground truth. This is achieved through offline profiling. First, we uniformly sample the packet loss rate between 0% and 20%. Based on the real-world packet loss measurement according to [21], the mean average packet loss is around 7% and the packet loss rate of over 80% traffic is within 20%. Then, given a packet loss and for each token number, we encode each video block with varying token numbers that are larger than the given value. We calculate the LPIPS improvement per token for all candidate token numbers. Finnally, we choose the one that is closest to the given token number and achieves the local optimum as the ground truth label, N_{gt}. When there is no packet loss, the original token number is the ground truth as there is no need to add additional tokens. During the training, we optimize the cross-entropy between the ground truth and the output of the neural network. To facilitate the training process, we normalize the token numbers with respect to the minimal values. We train the token predictor on a 4090-GPU server, and it takes 16 hours to run 300 epochs to converge to a relatively low and stable entropy loss.

GenStream can be analogous to FEC approaches to some extent. Adding more tokens can increase robustness to the packet loss, like adding parity packets in FEC. However, the added tokens have the "semantic" meaning which can also improve the quality of reconstructed video. Compared to Reparo, which uses image tokenizer, GenStream has three advantages. First, GenStream applies the video tokenizer to video streaming to improve compression efficiency. Second, while Reparo fine-tunes the image codecs to handle token loss and drop, it cannot react to the network dynamics proactively.

4 Evaluation

4.1 Experiment Setup

Dataset. We use broadband traces from FCC [8] (released in Jan. 2023), which contains bandwidth time series, to evaluate GenStream and other baseline schemes. For the video dataset, we use UCF101 dataset [23] for training the token predictor and fine-tuning the tokenizer. UCF101 is a realistic action video dataset, collected from YouTube, having 101 action categories with 13320 videos, each lasting tens of seconds. We use another video dataset from [4] for evaluation. This video dataset contains 62 video clips, where each video is 10-30 seconds long.

Video tokenizer. We adopt the video tokenizer as in [32] which employs a ViT encoder and decoder [2, 18], followed by a standard transformer block as in [26]. The resolution and FPS of the video are set as 256×256 and 30, respectively. Every 4 frames of video are encoded into one block with token numbers ranging from 640 to 3840. The codebook size of 64000 [18], meaning 16 bits represent token. Every 128 tokens are wrapped into one packet for transmission. A packet header that contains the information of frame index, packet index, and the token number index is added. We fine-tune the video tokenizer following a similar procedure in [25]: we implement a simulator pipeline that simulates the video encoding, packetization and decoding, and manually inject the packet loss rate within 20%. Then we jointly fine-tune the encoder and decoder to minimize the weighted sum of LPIPS and MSE on a 4×A40-GPU server. It takes over 1 day to train the model for 2 epochs.

Baseline schemes. We consider three baseline schemes which are the most up-to-date: Grace [4] is a neural network-based solution that adopts NVC [15], Reparo [12] adopts the VQGAN [7] as the image tokenizer to compress each frame of the video independently. Tambur [21] is one of the FEC approaches that uses the streaming code and adopts VP9 video codec. The target bitrate is selected from $\{500, 1000, 2000\}$ kbps. We use the video conference platform Ringmaster [1] provided by the authors to evaluate Tambur without changing any hyperparameters. We modified the decoder logic in Ringmaster so that we can save the decoded frame on the fly to calculate the frame-level metric like LPIPS.

Testbed. We benchmark all schemes using emulation. For Tambur, we run the experiment on two Linux servers running Ubuntu 20.04. The machine is equipped with two Intel Xeon E5-2660 v3 CPUs (20 physical cores, 40 threads total) and 164 GB of RAM. For other schemes, we first run the WebRTC [28] between the sender and receiver with emulated network conditions using Mahimahi [20]. Specifically, the bandwidth is from the real-world dataset. The packet loss is simulated using Gilbert-Elliott (GE) mode [9], which is widely used in the existing work [12, 21]. The GE loss model is a standard loss model which is a Markov model with two states: "good" and "bad," each with an associated probability of packet loss. The transition probability from "good" channel to "bad" channel is 0.068 while the reverse is

0.825. The packet loss rates at “good"" and “bad"" states are 0.04 and 0.525, respectively. Then generated network traces from the WebRTC are used to evaluate GenStream, Grace, and Reparo. The video tokenizer is run on a 4090-GPU Linux server.

4.2 Evaluation Results

In this part, we show our preliminary results and the advantage of GenStream, and the insights from employing the video tokenizer for a streaming system.

Compression efficiency. Fig. 6 shows the trade-off between the video quality and the bitrate of different schemes. It is worth noting that LPIPS is measured end-to-end, which takes network conditions into account. GenStream shows a significant performance gain over all baseline schemes, statistically given the target bitrate of 0.24 Mbps. The average LPIPS of GenStream is 0.0404, which achieves 59%, 38%, and 45% quality improvement compared to Grace, Reparo, and Tambur. While Grace and Reparo fine-tune the video codec to improve its robustness to packet loss, they may fail to handle varying packet loss patterns that are unseen before during the operation. Instead, GenStream leverages *token predictor* to improve its adaptability to the network conditions. For Tambur, there is a larger portion of data samples whose LPIPS is small (i.e., < 0.06) compared to the Rerapo. This is mainly because Tambur employs FEC that exist s cliff effect where the packet loss is within its capacity, then the quality would be excellent. However, if FEC cannot handle the packet loss anymore, then the quality of the recovered video would significantly degrade.

We also find that the video quality of generative codecs like GenStream (std dev 0.0295) and Reparo (0.0220) exhibits less variation compared to the traditional codecs Grace (0.0569) and Tambur (0.0604). This is hard for traditional codecs to maintain a consistent video quality given a target bitrate due to content complexity and the complex logics of codecs.

Samples. In this part, we want to show an example (time series) of token adaptation to the bandwidth fluctuation and packet loss patterns. Together with the video frame samples. Some discussion be added here. TO DO.

Latency and computation cost. In this part, we show main concerns of employing the video tokenizer for video streaming. The decoding frame per second can reach to 30 on 4090 GPU. While the value is quite low compared to the traditional codecs like h264/265, achieving 30 FPS for the video streaming is still useful for some downstream applications like video analytics. Furthermore, such FPS is comparable to traditional neural codecs [3].

4.3 Correlation Analysis

Video contents. Fig. 7a illustrates that GenStream achieves varying performance for different videos. For instance, video-4 is a high-resolution video showing a clear baby face, while video-0 is a dark environment with a person riding a bike from a long distance. Comparing these two videos, video-4 has much more detail and the whole scene is more contrastive, while video-0 contains less information. In this way, GenStream can have better performance on video-0. When the packet loss exists, the quality degradation for complex video will be larger than that of simple videos.

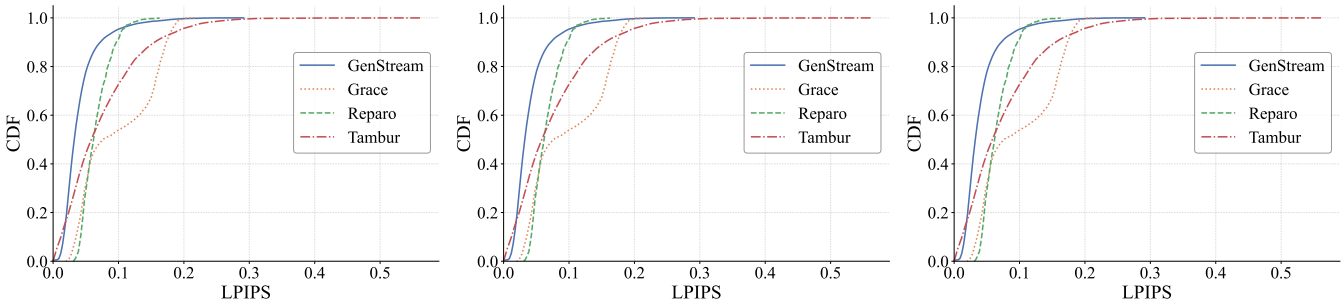
Token number. Fig. 7b illustrates the impact of token number on the video quality. As we can see, there is a marginal improvement when the token number is over 2176. Counter-intuitively, video compressed with fewer token numbers is more robust to the packet loss. It is interesting to see that the performance is not only affected by the proportion of tokens lost to the total number, but also by the absolute number of lost tokens.

Video blocks. Fig. 7c shows the impact of the video blocks. Specifically, the transform at the decoder side takes one video chunk that contains 4 video blocks, 16 frames in total, for sequential decoding. In this section, we run a controlled experiment that applies distinct packet-loss rates to block groups $n = 0, 1, 2$, and 3, respectively, and independently, where each block group n contains all blocks whose index modulo 4 equals n. Once get the decoded video frame, we calculate the LPIPS. Interestingly, we found that the first video block in the video chunk is the most important to maintain video quality. The possible reason is that the first block contains a lot of conditioning information to help with the reconstruction of future frames when using a transformer-based decoder. This also provides the guidance to explicitly bias the token generation process to protect those “important" blocks.

5 Discussion

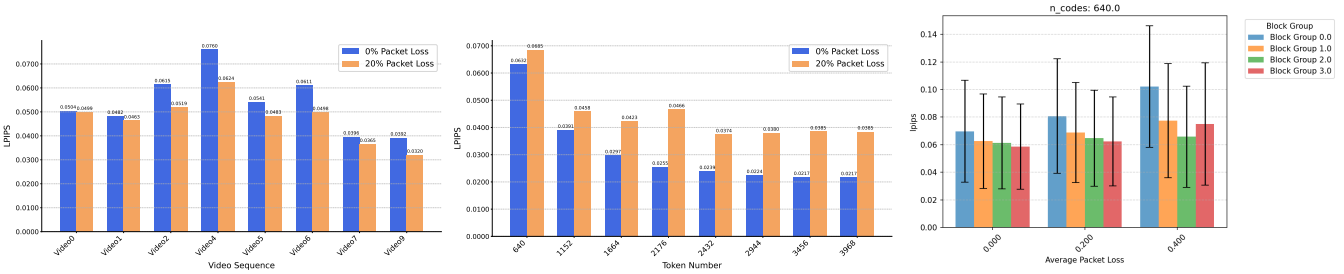
In this part, we discuss the future work for the project. The final goal is to make a short paper submission to a venue like HotNet2025. Currently, we have preliminary results that prove our idea of employing the video tokenizer and token predictor. We plan to scale the experiments in several aspects.

First, we plan to run GenStream with different real-world datasets collected from different scenarios. Satellite and underwater networks are the two choices. As for the packet loss trace, we are now collecting the packet loss pattern for zoom application in the real world using Wireshark tools [19]. Second, we still have to spend some time optimizing the experiment and analyzing the results to discover the insights. For example, use a visualized example to show the logic of token adaptation; explain why more token performs worse than



(a) Mobile Network. Bitrate is 0.24Mbps. (b) Satellite[TO DO]. (c) Underwater[TO DO].

Figure 6: End-to-end performance comparison on different network environments.



(a) Impact of video. (b) Impact of token number. (c) Impact of video block.

Figure 7: Correlation analysis.

fewer tokens given the same packet loss rate. Compare short and long videos. The packet loss pattern generated for training and fine-tuning can be further improved. Third, some ablation studies and analysis regarding the token predictor should be added.

6 Conclusion

References

[1] 2023. Ringmaster. https://github.com/microsoft/ringmaster.

[2] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. 2021. ViViT: A Video Vision Transformer. In 2021 IEEE/CVF International Conference on Computer Vision (ICCV). 6816–6826. https://doi.org/10.1109/ICCV48922.2021.00676

[3] Bo Chen, Zhisheng Yan, Yinjie Zhang, Zhe Yang, and Klara Nahrstedt. 2024. {LiFteR}: Unleash Learned Codecs in Video Streaming with Loose Frame Referencing. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24). 533–548.

[4] Yihua Cheng, Ziyi Zhang, Hanchen Li, Anton Arapin, Yue Zhang, Qizheng Zhang, Yuhan Liu, Kuntai Du, Xu Zhang, Francis Y Yan, et al. 2024. {GRACE}:{Loss-Resilient}{Real-Time} video through neural codecs. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24). 509–531.

[5] Cisco. 2022. Cisco Visual Networking Index: Forecast and Trends, 2017–2022. https://www.cisco.com/c/dam/m/en_us/solutions/service-provider/vni-forecast-highlights/pdf/Global_Device_Growth_Traffic_Profiles.pdf. Accessed: 2025-05-14.

[6] Mallesham Dasari, Kumara Kahatapitiya, Samir R Das, Aruna Balasubramanian, and Dimitris Samaras. 2022. Swift: Adaptive video streaming with layered neural codecs. In 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 103–118.

[7] Patrick Esser, Robin Rombach, and Bjorn Ommer. 2021. Taming transformers for high-resolution image synthesis. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 12873–12883.

[8] Federal Communications Commission. 2024. Measuring Broadband America: Raw Data Releases. https://www.fcc.gov/oet/mba/raw-data-releases. Accessed: 2025-05-13.

[9] Elliott W. Gilbert. 1960. Capacity of a burst-noise channel. Bell System Technical Journal 39, 5 (1960), 1253–1265.

[10] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. In Advances in Neural Information Processing Systems, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger (Eds.), Vol. 27. Curran Associates, Inc.

[11] Jiahao Li, Bin Li, and Yan Lu. 2023. Neural video compression with diverse contexts. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 22616–22626.

[12] Tianhong Li, Vibhaalakshmi Sivaraman, Pantea Karimi, Lijie Fan, Mohammad Alizadeh, and Dina Katabi. 2023. Reparo: Loss-resilient generative codec for video conferencing. arXiv preprint arXiv:2305.14135 (2023).

[13] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. 2020. Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 359–376. https://doi.org/10.1145/3387514.3405874

[14] Ruoshi Liu, Huy Ha, Mengxue Hou, Shuran Song, and Carl Vondrick. 2024. Self-Improving Autonomous Underwater Manipulation. arXiv preprint arXiv:2410.18969 (2024).

[15] Guo Lu, Wanli Ouyang, Dong Xu, Xiaoyun Zhang, Chunlei Cai, and Zhiyong Gao. 2019. DVC: An end-to-end deep video compression framework. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 11006–11015.

[16] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with pensieve. In Proceedings of the

conference of the ACM special interest group on data communication. 197–210.

[17] Qi Mao, Tinghan Yang, Yinuo Zhang, Zijian Wang, Meng Wang, Shiqi Wang, Libiao Jin, and Siwei Ma. 2024. Extreme image compression using fine-tuned vqgans. In 2024 Data Compression Conference (DCC). IEEE, 203–212.

[18] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. 2023. Finite scalar quantization: Vq-vae made simple. arXiv preprint arXiv:2309.15505 (2023).

[19] Oliver Michel, Satadal Sengupta, Hyojoon Kim, Ravi Netravali, and Jennifer Rexford. 2022. Enabling passive measurement of zoom performance in production networks. In Proceedings of the 22nd ACM internet measurement conference. 244–260.

[20] MIT Computer Science and Artificial Intelligence Laboratory. 2024. Mahimahi: Lightweight Record-and-Replay for HTTP. http://mahimahi.mit.edu/. Accessed: 2025-05-13.

[21] Michael Rudow, Francis Y Yan, Abhishek Kumar, Ganesh Ananthanarayanan, Martin Ellis, and KV Rashmi. 2023. Tambur: Efficient loss recovery for videoconferencing via streaming codes. In 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). 953–971.

[22] Shibani Santurkar, David Budden, and Nir Shavit. 2018. Generative compression. In 2018 Picture Coding Symposium (PCS). IEEE, 258–262.

[23] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. 2012. UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild. arXiv:1212.0402 [cs.CV] https://arxiv.org/abs/1212.0402

[24] Bill Tao, Maleeha Masood, Indranil Gupta, and Deepak Vasisht. 2023. Transmitting, Fast and Slow: Scheduling Satellite Traffic through Space and Time. In Proceedings of the 29th Annual International Conference on Mobile Computing and Networking (Madrid, Spain) (ACM MobiCom '23). Association for Computing Machinery, New York, NY, USA, Article 24, 15 pages. https://doi.org/10.1145/3570361.3592521

[25] Beitong Tian, Lingzhi Zhao, Bo Chen, Mingyuan Wu, Haozhen Zheng, Deepak Vasisht, Francis Y. Yan, and Klara Nahrstedt. 2025. AquaScope: Reliable Underwater Image Transmission on Mobile Devices. arXiv:2502.10891 [cs.NI] https://arxiv.org/abs/2502.10891

[26] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023).

[27] Deepak Vasisht, Jayanth Shenoy, and Ranveer Chandra. 2021. L2D2: Low latency distributed downlink for LEO satellites. In Proceedings of the 2021 ACM SIGCOMM 2021 Conference. 151–164.

[28] WebRTC Project. 2024. WebRTC: Real-Time Communication for the Web. https://webrtc.org/. Accessed: 2025-05-13.

[29] Zhengxu Xia, Yajie Zhou, Francis Y Yan, and Junchen Jiang. 2022. Genet: Automatic curriculum generation for learning adaptation in networking. In Proceedings of the ACM SIGCOMM 2022 Conference. 397–413.

[30] Yaxiong Xie and Kyle Jamieson. 2022. Ng-scope: Fine-grained telemetry for nextg cellular networks. Proceedings of the ACM on Measurement and Analysis of Computing Systems 6, 1 (2022), 1–26.

[31] Francis Y. Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. 2019. Learning in situ: a randomized experiment in video streaming. arXiv:1906.01113 [cs.NI] https://arxiv.org/abs/1906.01113

[32] Wilson Yan, Volodymyr Mnih, Aleksandra Faust, Matei Zaharia, Pieter Abbeel, and Hao Liu. 2024. Elastictok: Adaptive tokenization for image and video. arXiv preprint arXiv:2410.08368 (2024).

[33] Qihang Yu, Mark Weber, Xueqing Deng, Xiaohui Shen, Daniel Cremers, and Liang-Chieh Chen. 2024. An image is worth 32 tokens for reconstruction and generation. Advances in Neural Information Processing Systems 37 (2024), 128940–128966.

[34] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 586–595.